My History with the Wang 600

My introduction to the Wang 600 architecture actually started with a Wang 520 when I began High School. The 520 is physically a little smaller and had a Nixie Tube display instead of the Panaplex 9-segment display. It was missing some keys, and perhaps some features. The school also had a mark-sense card reader for it. The next year the school upgraded to a Wang 600, keeping the same card reader (which somewhat limited its use). I spent many hours discovering all I could about the internal workings of the machine, far beyond the documented features.

The Wang 500 series still used core memory and, as mentioned before, Nixie tubes for the display. The machines also used a special type of ROM for the microcode, the “wire weave” design invented by Dr. Wang.

The Wang 600 series used semiconductor RAM and the 9-segment Panaplex display (still neon gas discharge, though). As later discovered, some of the Wang 600 units had semiconductor ROM for the microcode, mask-programmed by the manufacturer (not field-programmable nor erasable).

About 6 years out of high school, I picked up two Wang 600 units from the Boeing Surplus Store. One was working, the other was not. I was not successful at fixing the broken one, and even made a (unsuccessful) plea to the local Wang Laboratories service center to get schematics. I then started the gigantic task of tracing out rats-nest schematics of each board (and the backplane and other hardware). This gave me a very high-level idea of how the machine worked, but did not help me repair the broken one.

A couple years later, the working unit died also. At that point, I discovered that the working unit had semiconductor ROM and embarked on a journey to copy the microcode off and save it. The idea also formed in my mind that I could write a computer program to simulate the Wang 600 hardware and “run” this microcode in a completely virtual environment (on a microcomputer or even mini/mainframe). I chuckled at the similarities to the old Sci-Fi movie “Colossus of New York” where a scientist transplants his dead brother’s brain into – at first – an artificial environment with audio input/output only and later into an artificial body with sight and motion – and lasers shooting from his eyes.

I built some hardware that would read the ROMs (eleven 2Kx4 chips) and transmit the data out a UART, which I connected to an ASR-33 teletype. I then proceeded to transfer each ROM chip onto paper tape. Once I had the ROMs archived on paper tape, I then connected the teletype to my Kaypro 2X and loaded each one into a file on the Kaypro. From there I started to explore the microcode and devise a plan to build the simulator.

I soon found that I did not have enough detail to be able to write a fully-functional simulator. I was able to disassemble the microcode, though, and even printed that out on wide paper. I then set the project aside until such time as I could work on it again.

Fast-forward 25 years, to a time when the web had started to become a useful, searchable, repository of human knowledge. I discovered that Wang schematics had (mostly) been preserved and could now be downloaded (with perhaps questionable license). I started learning my way around the Wang schematics and found that my rats-nest drawings were actually pretty close, and my understanding of the principals of the architecture were also pretty close.

I began work on writing a simulator for the Wang hardware. I wrote this in “C” under Linux, but knew there would be challenges to making a user interface that was “pleasing”. My knowledge of Linux GUI software was limited, but enough to know that would be difficult. I had heard of JAVA having integrated GUI, and had wanted to learn JAVA so that was my back-pocket plan. But, I knew “C” so continued to write the simulator in “C”.

The first version of the “C” simulator had no input, and the only output was a crude implementation of the display refresh… it simply printed each display digit as it was refreshed, and used a ‘\r’ between refreshes to keep the output on the same line. Very crude, but when I ran the simulator and saw this “flickering” display of “0.000000000” appear on my screen, it was exciting. I then added a crude mechanism for inputting key codes and began testing things a bit more. This included the ability to set the RUN/LEARN/… switch. Watching the “display” switch between “0.00000000” and “0000 00 00” when changing the RUN/LEARN mode was another great moment.

I then set out to build a GUI that would give justice to the original Wang 600. This including learning JAVA, so it was quite the detour. Rather than discard the existing hardware simulation in “C”, I set out to use the “C” code and communicate with the JAVA GUI using stdio.

Being new to JAVA, and relatively fresh to object-oriented programming, I took short-cuts. I crammed the whole GUI (all objects) into one source file. The objects themselves were more of convenience rather than good design. But, I got a JAVA GUI working and interacting with a “C” simulator “back end”. Another big milestone.

I then began working on converting the “C” code to JAVA and integrating that into the JAVA GUI code. Once this was done, the Wang 600 simulator could be run entirely from a single JAR file.

As the long Minnesota winters will do, boredom set in and I decided to work on a Wang 700 version of the simulator, without really having any experience with the machines at all (I had seen them for sale at Boeing Surplus).

Since the 700 series all had the “wire weave” microcode ROM, it is considerably more risky to try and extract the microcode – poorly designed hardware to extract could easily fry the 40-gauge wire used there. I had found a crude copy of the microcode that an ingenious person had extracted using the Wang 700 hardware with parts of the CPU replaced with latches and HEX LED displays, and a button to advance to the next word. Each microcode word had to be manually transcribed from the HEX displays, which of course is error prone.

Armed with this microcode and the Wang 700 schematics, I worked on a simulator. This also involved “objectifying” the Wang 600 JAVA code, in order to share as much as possible.

The Wang 700 architecture is significantly different, although many of the same ideas were used. Unfortunately, there were transcription errors in the microcode I had acquired, so every problem had to be considered as either a hardware-interpretation mistake (simulator bug) or a transcription error (microcode bug). After fixing several transcription errors and getting the simulation aligned with the hardware, the task of thoroughly scrubbing the microcode for transcription errors began. This required learning many of the obscure features of the Wang 700 and executing them to see if the expected results were obtained. After much microcode tracing, it appears that all transcriptions errors have been fixed.

Finding transcription errors was no small task. One has to recognize that an operation failed, then gather traces of the microcode when executing the operation (and failing), and locating where the microcode is derailed and determining what the correct action should be. This is compounded by the fact that the microcode is “spaghetti” code by default – each microcode instruction normally has the address of the next instruction to execute, except as modified by “case” (and other conditional) instructions. Without call/return, the code becomes even more convoluted as state variables must be used to ensure that shared code or loops are only executed when required. Suffice to say, it is fortunate that the number of transcription errors was relatively small. However, one can never be 100% certain that all microcode instructions have been covered, and for all possible states/conditions.

I was then contacted by a collector about the Wang 1200 word processing machine, which is based on the Wang 600 CPU architecture. Armed with semiconductor ROM images extracted from real hardware, and more schematics, I was able to create a simulation of the Wang 1200 as well – albeit different since it is based on a Selectric typewriter user interface, instead of a calculator UI.